Group symbol: **2**

Team: **1**

Project title: **ListItUp**

Team members (*filled by PM, Team Leader*):

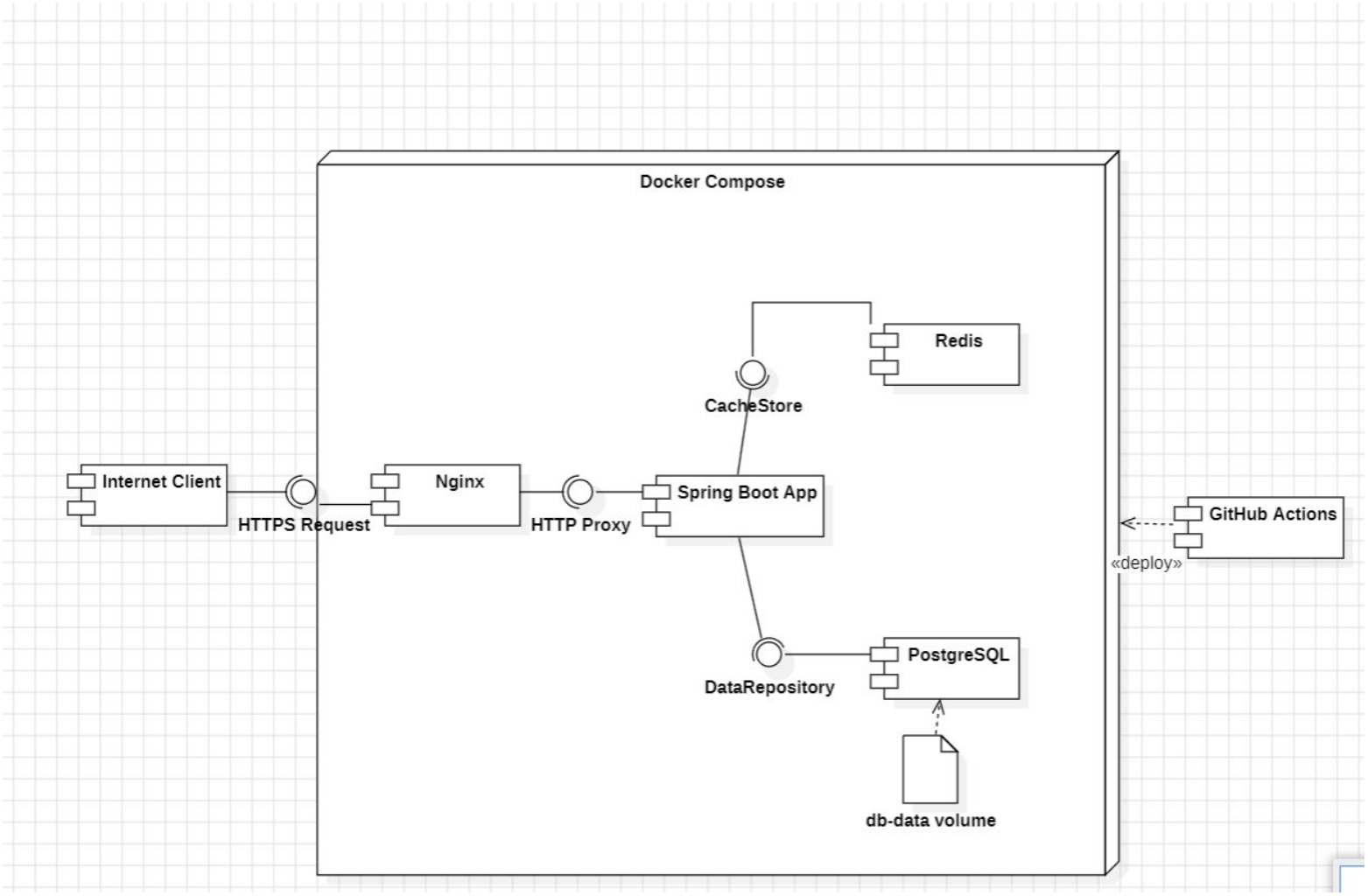
No	Name	Surname	Student ID	Role
1	Carmelo	Romero Pérez	293936	*PM, Team Leader*
2	Álvaro	Puebla Ruisánchez	293867	*Team member*
3	Carmen	Gutiérrez Sánchez	293906	*Team member*
4	David	Sosa Domínguez	293885	*Team member*
5	Diego	García Agrelo	293939	*Team member*
6	Miryam	Merchán León	293907	*Team member*

3. Design **(S3)**

3.1. Logical Software Architecture

The logical architecture aspect of the system should be present as a Component Diagram expressed in UML 2.5.

The logical architecture of ListItUp follows a layered, container-based model decomposed into six primary components orchestrated via Docker Compose. All external traffic enters through Nginx, which terminates TLS and routes requests to the Spring Boot application. The database is isolated in its own container. The CI/CD pipeline runs on GitHub Actions and, while not part of the runtime topology, is shown for completeness.



3.2. Business Logic Model

3.2.1. Behavioural Model

In this section present:

- *state diagram for whole the system and*

The state diagram below describes the lifecycle of a List created by an user within the ListItUp system.

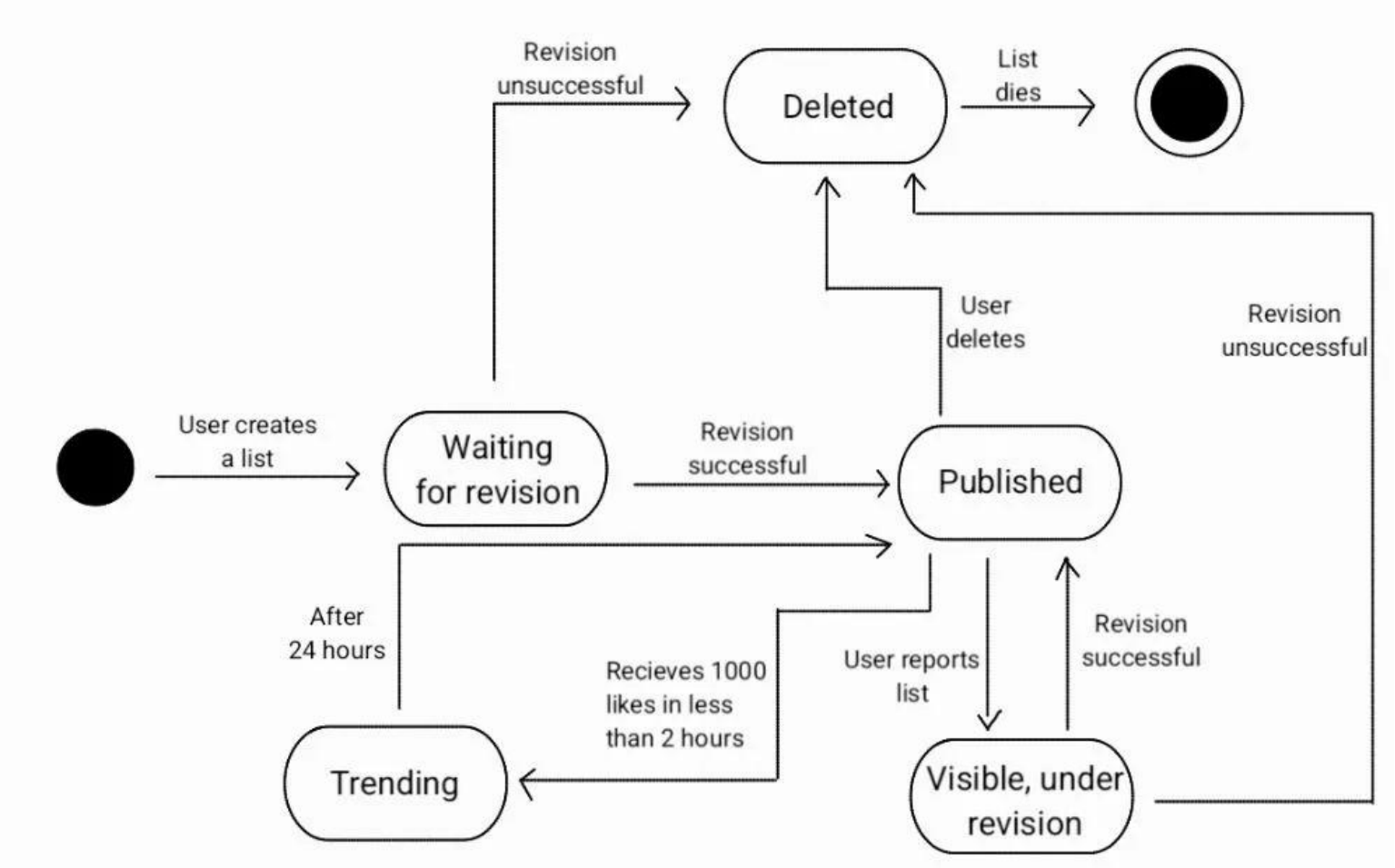
A list begins unpublished waiting for revision, until the system verifies it, anyone can see it. Once verification is completed, a list can be published or deleted depending on the results of the verification

If the list is published, other users can interact with it. If the list receives 1000 likes in less than 2 hours it becomes a trending list. When 24 hours has passed, the list becomes published again (existing the possibility to receive 1000 likes in less then 2 hours and be trending again).

A published list can be reported by other users, if so, it is still visible but under revision.

After that revision the list could be published as normally if the revision is successful or deleted if it is unsuccessful.

When a list is deleted it is cleared out of the system and dies.



- *behaviour of 2 crucial use cases using the UML 2.5 diagrams, namely sequence diagram, and activity diagram.*

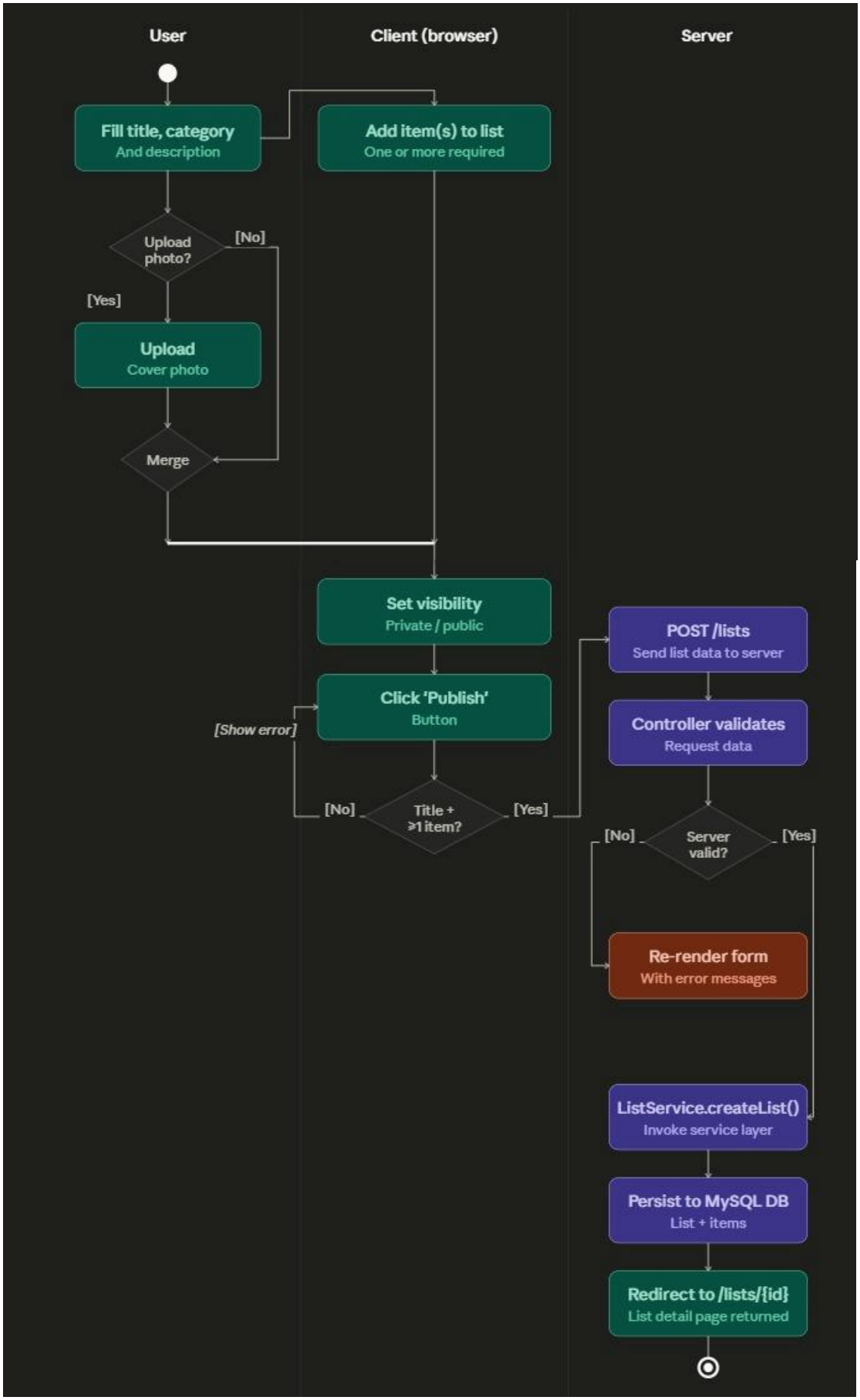
Use Case 1 - Create and Publish a List (Sequence & Activity Diagrams)

This use case covers the full creation flow from the user clicking '+ Create List' to the newly published list appearing in the discovery feed. It is one of the most critical flows as it is directly tied to the core business goal of making content creation effortless (target: ≥ 80% of pilot users able to create and publish without assistance).

Sequence Diagram - UC1: Create and Publish a List

UC01	Create and publish a list
Description	The user creates a new list with metadata and items, then publishes it to the system.
Prerequisites	The user has logged in and navigated to /lists/new.
Normal flow	
1	The user clicks 'Create List' and the browser requests GET /lists/new.
2	Nginx routes the request and Spring Boot Controller renders and returns the empty form.
3	The user fills in the list metadata (title, category, description) and adds items.
4	The user clicks 'Publish' and the browser sends POST /lists to the server.
5	The Controller validates the request data.
6	ListService.createList() is invoked and persists the list and its items to MySQL DB.
7	The server redirects the browser to /lists/{id} and the list detail page is returned.
Variant	
5.1	Client-side validation fails (missing title or no items): the browser shows an inline error and does not submit the form.
5.2	Server-side validation fails: the Controller re-renders the form with error messages.
3.1	The user optionally uploads a cover photo before adding items.
3.2	The user sets the visibility of the list to private or public.
6.1	ItemService persists each item individually and associates it with the new list.

Activity Diagram - UC1: Create and Publish a List



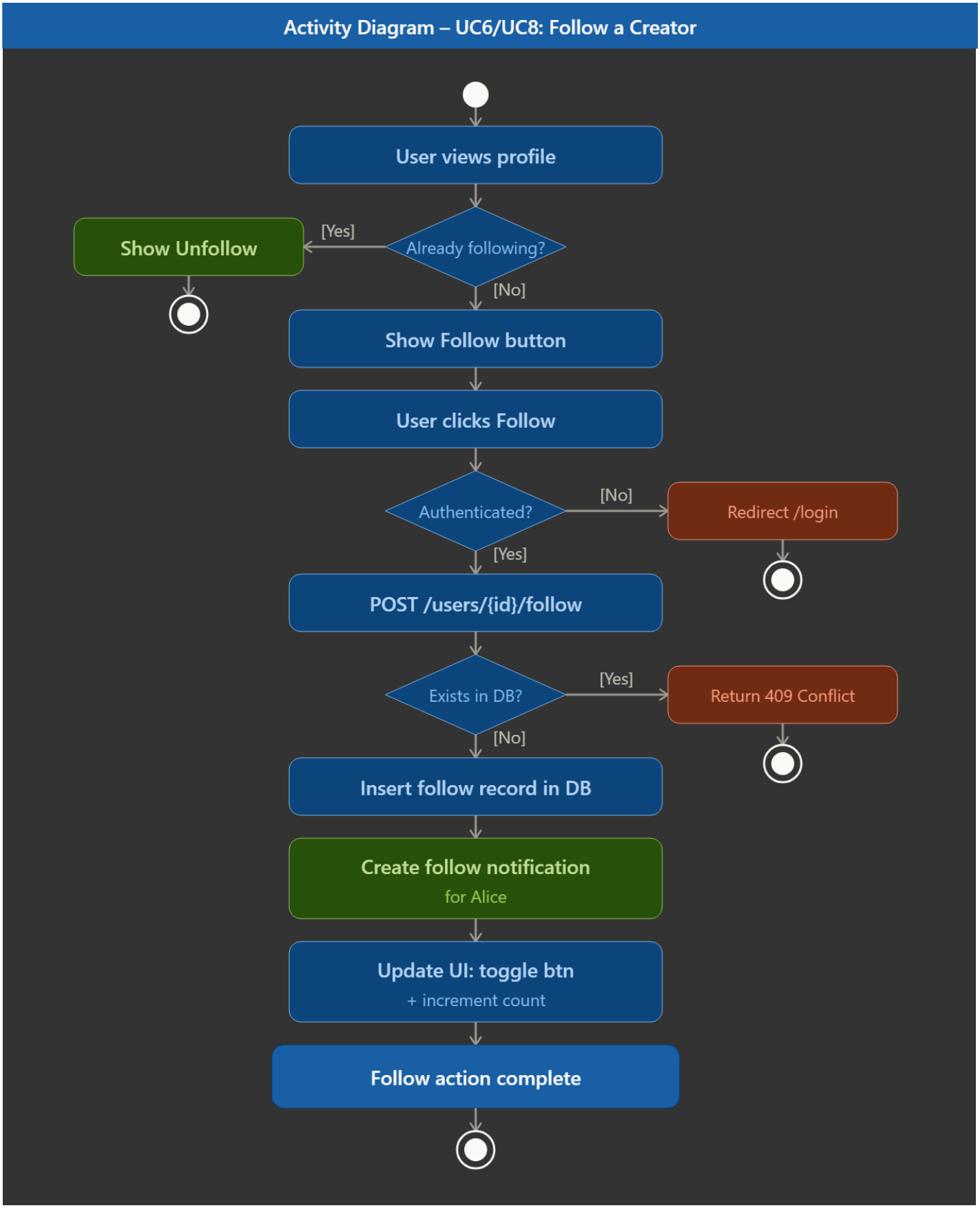
Use Case 2 - Follow a Creator and Receive Notification (Sequence & Activity Diagrams)

This use case covers the social follow action and the resulting in-app notification, directly tied to the business goal of growing an active community (target: ≥ 15% interaction rate among pilot users). It involves cross-service coordination between UserService and NotificationService.

Sequence Diagram - UC6/UC8: Follow a Creator

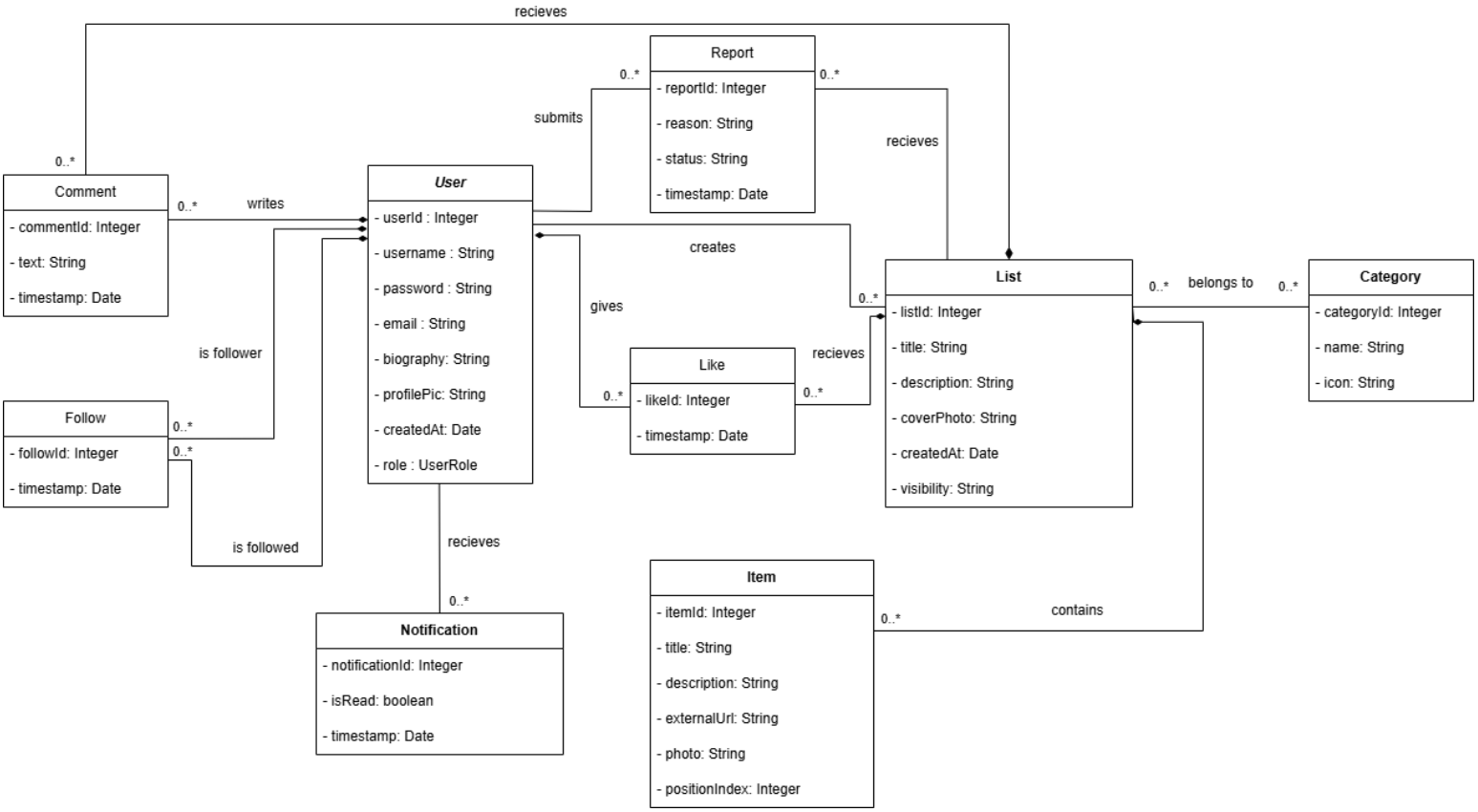
UC6 / UC8	Follow a Creator
Description	A logged-in user (Bob) follows another user (Alice) as a creator
Prerequisites	The user (Bob) has logged in and is viewing Alice's profile
Normal flow	
1	Bob clicks the Follow button on Alice's profile.
2	The browser sends POST /users/{aliceId}/follow to the server.
3	Nginx forwards the request to the Spring Boot Controller.
4	The controller verifies Bob's authentication.
5	The controller calls UserService.followUser(bob, alice).
6	UserService inserts a record into the follow table in MySQL.
7	UserService calls NotificationService.createFollowNotification(alice, bob).
8	NotificationService inserts a notification record into MySQL.
9	The system returns 200 OK back through Nginx to the browser.
10	The browser updates the Follow button state and follower count optimistically.
Variant	
4a	If Bob is not authenticated, the controller redirects to /login.
6a	If the follow record already exists, the system returns 409 Conflict.

Activity Diagram - UC6/UC8: Follow a Creator (Alice)



3.2.2. Structural Model

Prepare the model as class diagram expressed in UML 2.5. Use natural language to describe the semantics of models' elements. Besides, use the object diagram to present an instance of the model at the time of system execution.



Semantics of model elements:

- User - central entity representing any registered account. The role attribute (STANDARD, VERIFIED, ADMIN) discriminates behaviour: Verified Creators gain analytics access; Admins gain moderation access. Login is exclusively via OAuth 2.0; no platform password is stored.
- CuratedList - a named, ordered collection of Items created by a User. The isPinned flag is only meaningful for Verified Creators and allows one list to be pinned on their profile page. The viewCount is an atomic counter incremented by the service layer on public page loads.
- Item - a single entry within a CuratedList carrying a title, description, optional photo, external URL, and a positionIndex for display ordering. Items cascade-delete with their parent list.
- Category - a read-only lookup table (seeded by the team) used to classify lists and power the discovery feed filters.
- Follow - a join entity representing a directed social edge from follower (User) to following (User). A unique constraint prevents duplicate follows; a check constraint prevents self-follow.
- Like / SavedList - thin join entities linking a User to a CuratedList, both with a unique (userId, listId) constraint to prevent duplicates.
- Comment - a textual contribution by a User on a CuratedList, cascade-deleted when either the author or the list is removed.
- Notification - a system-generated message to a recipient User, optionally referencing a triggerUser and a relatedList. The notifType enum classifies events: LIKE, COMMENT, FOLLOW, NEW_POST.

- Report - a user-submitted moderation request. It targets either a CuratedList or a Comment via nullable foreign keys. The status enum tracks the review lifecycle: OPEN → REVIEWED → RESOLVED.

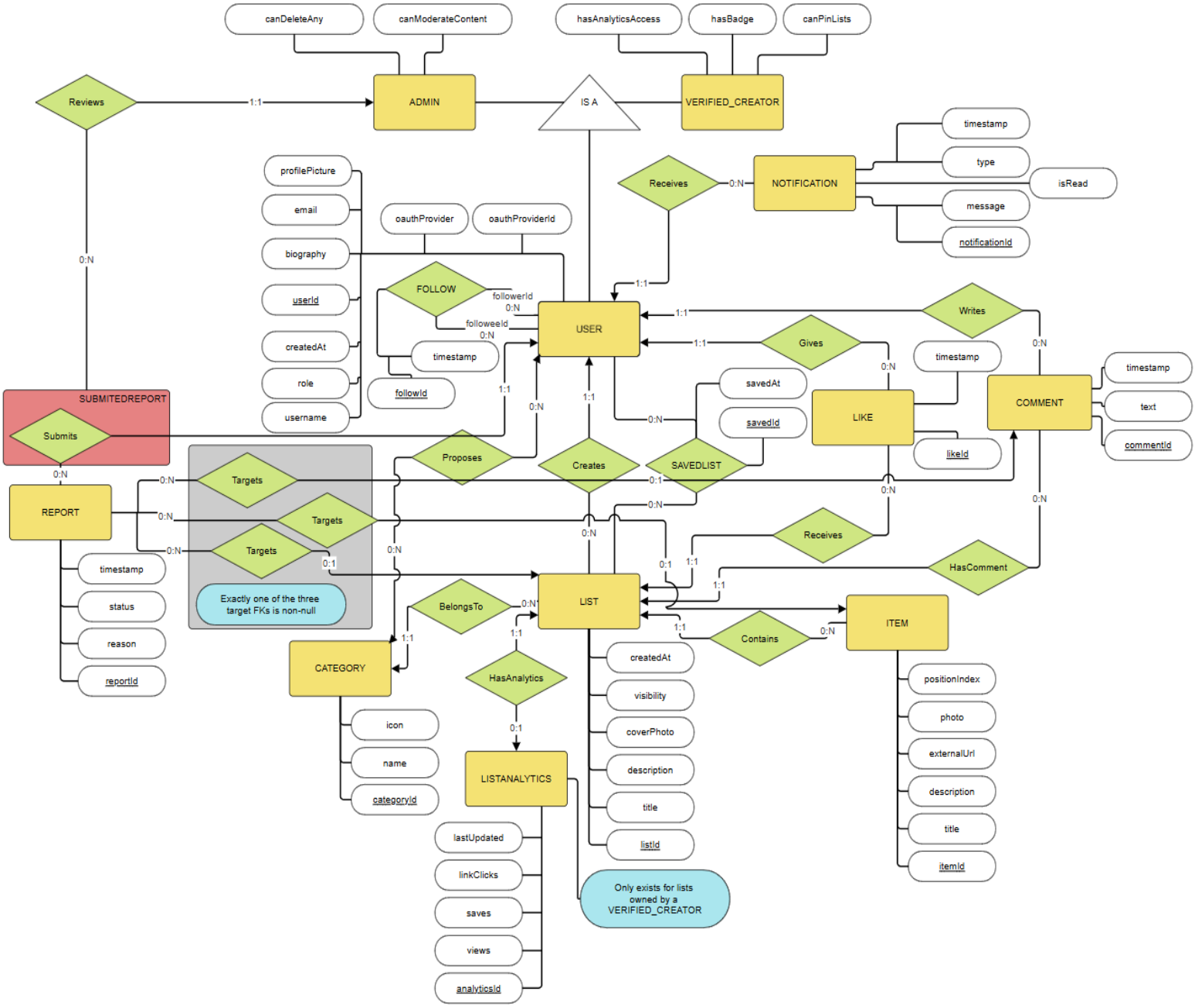
Object Diagram - Execution Snapshot

The following instance diagram presents a concrete snapshot of the system at a specific moment in time. Scenario: User 'alice' (VERIFIED) has published two lists. User 'bob' (STANDARD) follows alice, has liked her first list, and left a comment. Alice has one unread notification.

3.3. Database Model

3.3.1. Conceptual Model

The model contains the definition of entities and relationships among them in terms of the database (persistence). You should present it as a diagram and description of key elements in natural language.



This model represents a social list-curation platform where users build and share ranked lists about anything. It includes specialization hierarchies for users and supports complex social interactions, content moderation, and creator analytics. The model ensures data consistency across list creation, item management, social engagement, and administrative oversight.

1. Entities

User

Represents any person registered in the system. Key attributes include a unique identifier (userId), username, email, authentication provider (OAuth: Google or GitHub), and account creation date. Notably, no passwords are stored - the system uses exclusively OAuth 2.0 for authentication (NF7 compliance). This entity functions as the parent for two specialized roles: VerifiedCreator and Admin.

VerifiedCreator

A specialized type of user with elevated privileges for content creation and analytics. Additional attributes include flags for verification badge display (hasBadge), the ability to pin lists to profile (canPinLists), and access to the analytics dashboard (hasAnalyticsAccess). Verified creators gain insight into their list performance through the ListAnalytics entity.

Admin

Another specialization of the User entity, representing platform moderators and staff. Attributes include permissions to moderate content (canModerateContent) and delete any user-generated content (canDeleteAny). Admins oversee the moderation workflow and review content reports submitted by users.

List

Represents a curated collection of items built and owned by a user. Key attributes include a unique identifier (listId), the creator's reference (creatorId FK), title, description, cover photo URL, visibility status (public or private), and creation timestamp. Each list belongs to exactly one category and may contain multiple items. Lists serve as the primary content unit of the platform.

Item

Represents a single entry within a list. Attributes include a unique identifier (itemId), the parent list reference (listId FK), title, description, an external URL (enriched with Open Graph metadata), photo URL, and a position index to preserve ordering. Items are always subordinate to a list and cannot exist independently.

Category

Represents a topic or classification for lists. Attributes include a unique identifier (categoryId), a name (unique), and an icon identifier. Categories help users discover lists by topic. Users can propose new categories through the CategoryProposal relationship, and administrators can create categories directly.

Comment

Represents a text comment left by a user on a list. Attributes include a unique identifier (commentId), the author reference (userId FK), the target list reference (listId FK), the comment text, and a creation timestamp. Comments enable social discourse around lists and are moderated for compliance (FR5).

Like

Represents a user's appreciation for a list. Attributes include a unique identifier (likeId), the user reference (userId FK), the list reference (listId FK), and a timestamp. Each user can like a list at most once, enforced via a UNIQUE constraint on (userId, listId). Likes serve as both a form of engagement and a ranking signal.

Follow

A self-referential relationship entity representing one user following another. Attributes include a unique identifier (followId), follower reference (followerId FK → USER), followee reference (followeeId FK → USER), and a timestamp. A user cannot follow themselves (enforced via CHECK constraint). This relationship powers the personalized feed and social graph discovery (UC8).

SavedList

A relationship entity representing a user's personal collection of saved (bookmarked) lists. Attributes include a unique identifier (savedId), the user reference (userId FK), the list reference (listId FK), and a save timestamp. Each user can save a list at most once, enforced via UNIQUE(userId, listId). Saved lists enable personal curation and are included in user profiles.

CategoryProposal

A relationship entity capturing user suggestions for new categories. Attributes include a unique identifier (proposalId), the proposing user reference (userId FK), the category reference (categoryId FK), and a proposal timestamp. Each user can propose a given category only once, enforced via UNIQUE(userId, categoryId). This enables crowdsourced category expansion while maintaining control over the category namespace.

Report

Represents a moderation action where a user flags content as violating platform guidelines. Attributes include a unique identifier (reportId), the submitter reference (submittedByUserId FK, nullable - SET NULL on user deletion to preserve moderation history), the reviewing admin reference (reviewedByAdminId FK, nullable), three optional target references (targetListId, targetItemId, targetCommentId - exactly one must be non-null to enforce disjoint targeting), a reason text, a status (open, reviewed, or resolved), and a timestamp. Reports enable community-driven content moderation (FR5).

Notification

Represents an in-app or email alert sent to a user. Attributes include a unique identifier (notificationId), the recipient reference (receiverUserId FK), a type (ENUM: like, comment, new_post, saved_list_update), a message text, an isRead flag, and a timestamp. The system generates notifications automatically when specified events occur (FR6): when someone likes a user's list, comments on it, follows the user, or when a saved list is updated.

ListAnalytics

A weak entity that stores aggregated performance metrics for lists owned by verified creators. Attributes include a unique identifier (analyticsId), the list reference (listId FK, UNIQUE ensuring 1:1), counters for views, saves, and link clicks, and a last updated timestamp. This entity only exists for lists owned by VerifiedCreator accounts and is updated daily (FR7). It provides creators with insight into their content's reach and engagement.

---

2. Relationships

Creates (User → List)

Cardinality: 1:N - One user creates many lists; each list is created by exactly one user

Participation: Total on List side (every list must have a creator); partial on User side

Delete Rule: ON CASCADE DELETE (GDPR compliance - user account deletion removes all personal lists)

Implementation: Indexed on creatorId for efficient querying of user lists

Notes: Enforces content ownership and enables user-specific list retrieval

Contains (List → Item)

Cardinality: 1:N - One list contains many items; each item belongs to exactly one list

Participation: Total on Item side (items cannot exist without a list); partial on List side

Delete Rule: ON CASCADE DELETE (orphaned items are automatically removed)

Implementation: Indexed on listId for ordering and pagination

Notes: Maintains structural integrity and prevents orphaned items

BelongsTo (List → Category)

Cardinality: N:1 - Many lists belong to one category; each list has exactly one category

Participation: Total on List side (categorization is mandatory); partial on Category side

Delete Rule: ON RESTRICT (prevents accidental category deletion; lists must be reassigned first)

Notes: Enables category-based discovery (UC6) and browsing

Writes (User → Comment)

Cardinality: 1:N - One user writes many comments; each comment is written by exactly one user

Participation: Total on Comment side; partial on User side

Delete Rule: ON CASCADE DELETE (GDPR - user deletion removes all personal comments)

Implementation: Indexed on userId for user comment history

Notes: Preserves comment authorship for moderation and attribution

HasComment (List → Comment)

Cardinality: 1:N - One list receives many comments; each comment targets exactly one list

Participation: Partial on both sides (lists may have no comments; comments are always on a list)

Delete Rule: ON CASCADE DELETE (list deletion removes all associated comments)

Notes: Enables comment threading and moderation per list

Gives (User → Like)

Cardinality: 1:N - One user gives many likes; each like is given by exactly one user

Participation: Total on Like side; partial on User side

Delete Rule: ON CASCADE DELETE (GDPR - user deletion removes engagement history)

Constraint: UNIQUE(userId, listId) prevents duplicate likes by the same user

Implementation: Indexed on userId for feed algorithms; composite index on (userId, listId) for duplicate detection

Notes: Prevents abuse and provides engagement metrics

Receives (List → Like)

Cardinality: 1:N - One list receives many likes; each like targets exactly one list

Participation: Partial on both sides (lists may have no likes)

Delete Rule: ON CASCADE DELETE (list deletion removes all likes)

Implementation: Efficient counting for like-based ranking

Notes: Powers like counters and engagement-based ranking

FOLLOW (User ↔ User, self-referential M:N)

Cardinality: M:N - A user follows many; many users follow a user

Participation: Partial on both sides (users may follow no one and be followed by no one)

Delete Rule: ON CASCADE DELETE on both sides (GDPR - user deletion removes all follows)

Constraints: UNIQUE(followerId, followeeId) prevents duplicate follows; CHECK(followerId ≠ followeeId) prevents self-follows

Implementation: Composite indexes on (followerId), (followeeId) and (followerId, followeeId) for follower/following queries and feed generation

Notes: Forms the social graph; enables personalized feed delivery (FR4, UC6)

SavedList (User ↔ List, M:N join)

Cardinality: M:N - One user saves many lists; one list is saved by many users

Participation: Partial on both sides (users may save no lists; lists may be saved by no users)

Delete Rule: ON CASCADE DELETE on both sides (GDPR on user side; logical cleanup on list side)

Constraint: UNIQUE(userId, listId) prevents duplicate saves by the same user

Implementation: Indexes on (userId, savedAt DESC) for chronological retrieval; (listId) for counting total saves per list

Notes: Enables personal list collections (FR4, UC11) and save-based ranking signals

CategoryProposal (User ↔ Category, M:N join)

Cardinality: M:N - One user proposes many categories; one category can be proposed by many users

Participation: Partial on both sides (not all users propose; categories may be admin-created)

Delete Rule: ON CASCADE DELETE on both sides

Constraint: UNIQUE(userId, categoryId) prevents duplicate proposals

Implementation: Indexed on userId for user proposal history; (categoryId) for admin review queue

Notes: Enables crowdsourced category expansion while maintaining admin control

Submits (User → Report)

Cardinality: 1:N - One user submits many reports; each report is submitted by one user (or anonymously)

Participation: Partial on both sides

Delete Rule: SET NULL (not CASCADE) - Reports remain in the moderation queue even after user deletion to preserve operational history. This is a GDPR balance: the report is operational data, not personal data about the reporter.

Notes: Enables community-driven moderation (FR5, UC17)

Targets (Report → List, partial)

Cardinality: N:0..1 - Many reports may target the same list; a report targets at most one list

Participation: Partial on both sides (reports may not target lists; lists may be unreported)

Delete Rule: SET NULL (report history is preserved even if list is deleted)

Constraint: Part of disjoint targeting rule - exactly one of (targetListId, targetItemId, targetCommentId) is non-null

Notes: Enables content-specific moderation

Targets (Report → Item, partial)

Cardinality: N:0..1 - Many reports may target the same item; a report targets at most one item

Participation: Partial on both sides

Delete Rule: SET NULL

Constraint: Disjoint targeting (exactly one target is non-null)

Notes: Enables item-level moderation for inappropriate links or descriptions

Targets (Report → Comment, partial)

Cardinality: N:0..1 - Many reports may target the same comment; a report targets at most one comment

Participation: Partial on both sides

Delete Rule: SET NULL

Constraint: Disjoint targeting (exactly one target is non-null)

Notes: Enables comment-level moderation for abusive or spam content

Reviews (Admin → Report, aggregation)

Cardinality: 1:N - One admin reviews many reports; each report is reviewed by zero or one admin

Participation: Partial on both sides (not all admins review; not all reports are reviewed)

Delete Rule: SET NULL (if admin is deleted, the review assignment is cleared for reallocation)

Pattern: Aggregation - conceptually, Admin reviews the act of (User Submits REPORT). The aggregation models that reviews are actions taken on completed user submissions, not isolated entities.

Notes: Enables moderation workflows and audit trails (FR5, UC15-UC16)

Receives (User → Notification)

Cardinality: 1:N - One user receives many notifications; each notification is received by exactly one user

Participation: Total on Notification side (every notification targets a user); partial on User side

Delete Rule: ON CASCADE DELETE (GDPR - notifications are ephemeral activity data)

Implementation: Indexed on receiverUserId for inbox queries; timestamp for chronological ordering

Notes: Delivers alerts for likes, comments, follows, and saved list updates (FR6, UC13-UC14)

HasAnalytics (List → ListAnalytics, 1:1 weak)

Cardinality: 1:0..1 - Each list has at most one analytics record; each record belongs to exactly one list

Participation: Partial on List side (only VerifiedCreator lists have analytics); total on ListAnalytics side (each analytics record must have a list)

Delete Rule: ON CASCADE DELETE (when list is deleted, analytics are removed)

Constraint: UNIQUE(listId) ensures the 1:1 relationship when present

Implementation: Analytics records created only when list owner is promoted to VerifiedCreator (enforced at application layer)

Notes: Powers creator dashboard (FR7, UC12) with views, saves, and link click metrics

Specialization Hierarchies

User → VerifiedCreator / Admin - The system uses inheritance to represent different user roles and permissions under a single parent entity. Attributes specific to each role are stored in the base User table (single-table inheritance

strategy via role ENUM), simplifying Spring Boot JPA mapping and reducing join complexity.

Resource → Book / Room (not applicable to ListItUp) - This model does not include a Resource hierarchy; instead, it focuses on List and Item as the primary content entities.

3. General Behavior of the System

The model supports:

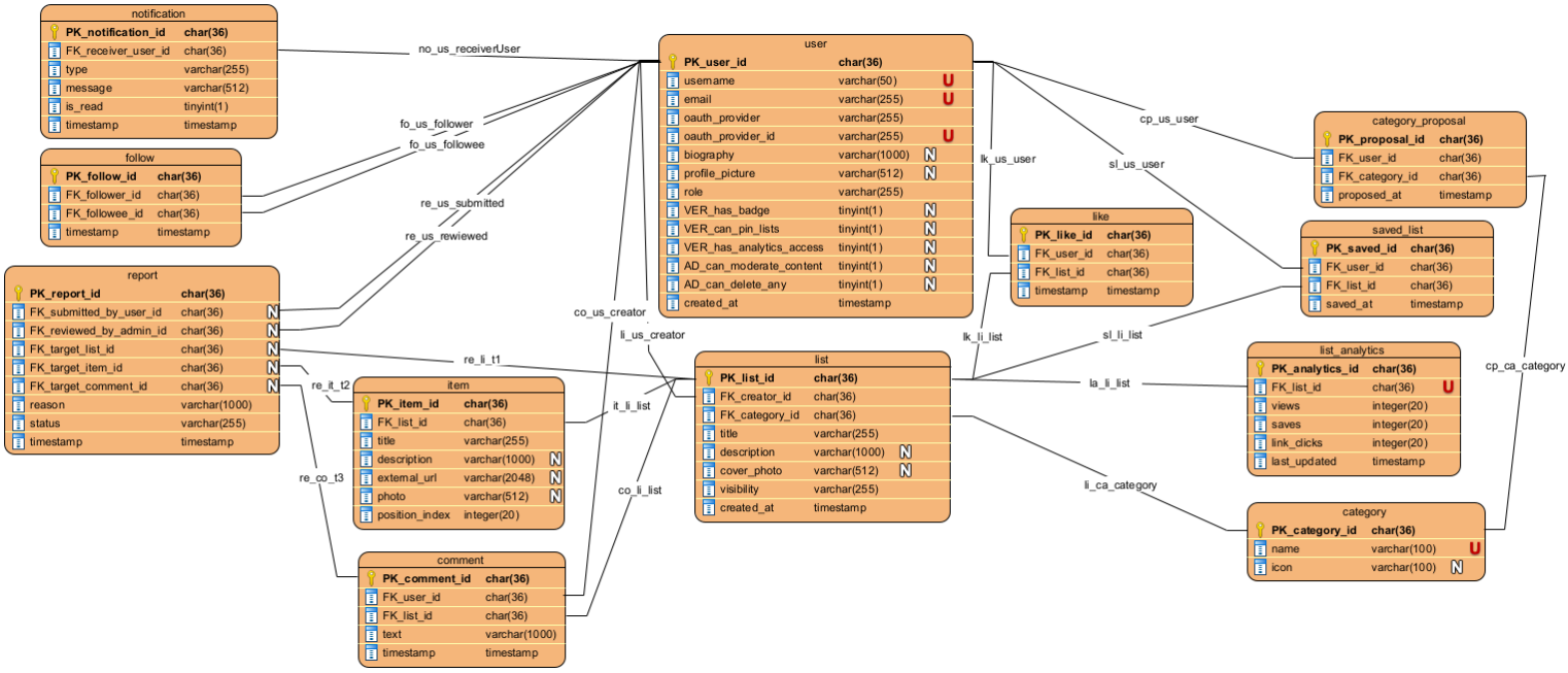
- Creation and publication of curated lists by any authenticated user (FR1)
- Item management within lists - adding, editing, reordering, and deletion (FR2)
- Category-based discovery and free-text search across public lists (FR3)
- Social engagement through follows, likes, comments, and list saves (FR4)
- Community-driven content moderation via user reports and admin reviews (FR5)
- In-app and email notifications for engagement and follower activity (FR6, should-have)
- Creator analytics dashboards for verified users, showing views, saves, and link clicks (FR7, should-have)
- GDPR compliance through cascading user deletion and data export capabilities (NF4)
- OAuth 2.0-exclusive authentication with no stored passwords (NF7)

Overall, the model ensures that all platform data is stored consistently while preserving clear separation between user roles, content types, and interaction patterns. Specialization hierarchies (User types) and relationship entities (Follow, SavedList, CategoryProposal) provide flexibility for the social and curation features central to the ListItUp experience.

3.3.2. Physical Model

The model presented as a diagram of the database that constitutes the implementation of the conceptual model in the selected database management system.

It is required to include an elaboration on the transformation of the conceptual model to the physical model. Description of transformation should include how elements like, e.g. inheritance and associations or other sophisticated modelling concepts have been handled.



The FK_follower_id : char(36) and the FK_followee_id : char(36) inside the table follow must be different as you cannot follow yourself: *UNIQUE (follower_id, followee_id) - no duplicate follows · CHECK (follower_id <> followee_id) - cannot follow yourself*

In the saved_list the user_id and the list_id must be unique: *UNIQUE (user_id, list_id) - a user saves a list only once*

In the category_proposal the user_id and the category_id must be unique: *UNIQUE (user_id, category_id) - a user proposes a given category only once*

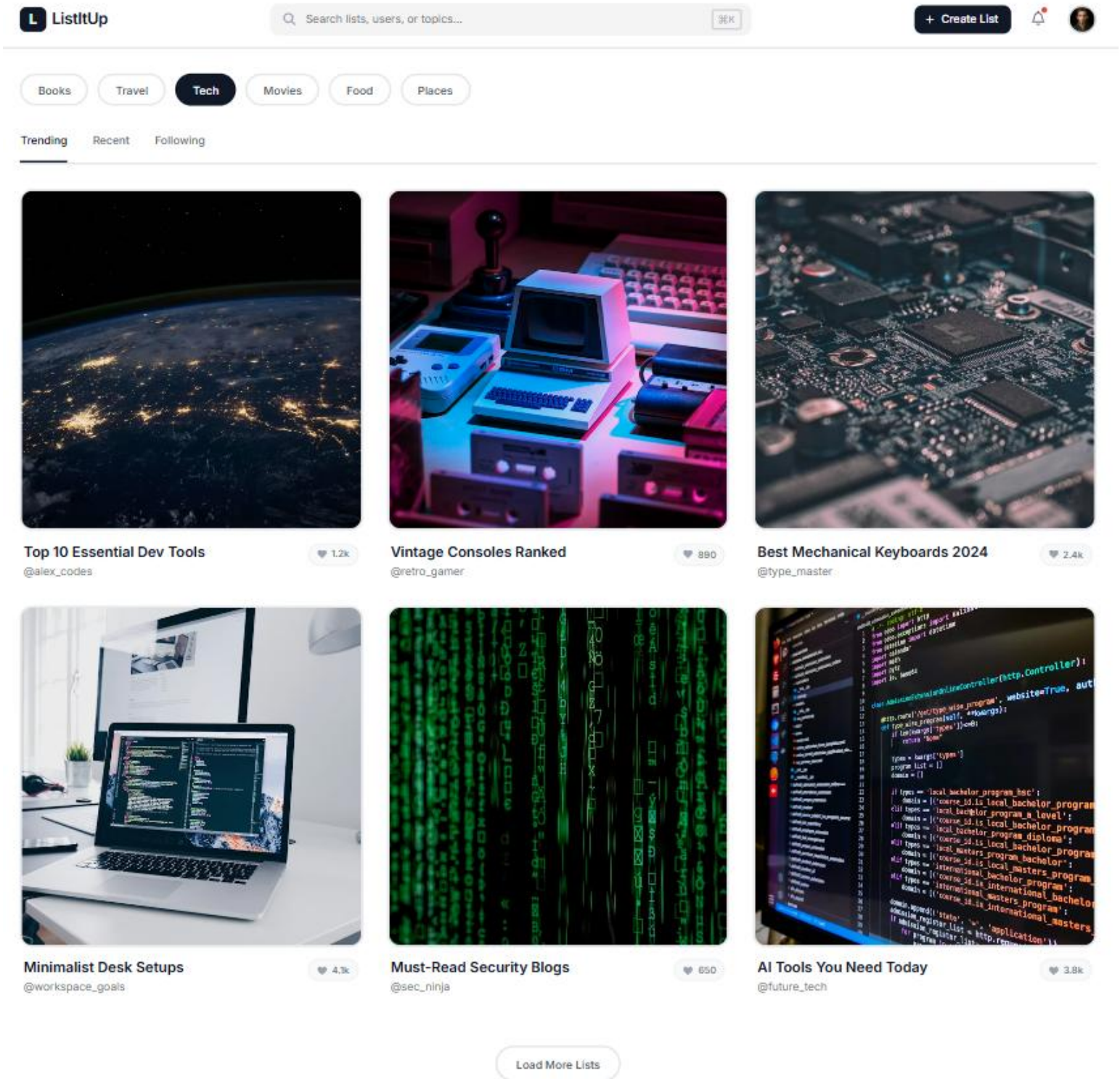
In the report check if targets have exactly 1 target: *CHECK: exactly one of (target_list_id, target_item_id, target_comment_id) is NOT NULL - enforced at app layer or via CHECK constraint*

3.4. User Interface Design

In this section, you should present the distribution of user interface components with events and their assignments to the behaviour specifications, which these elements initiate or take part.

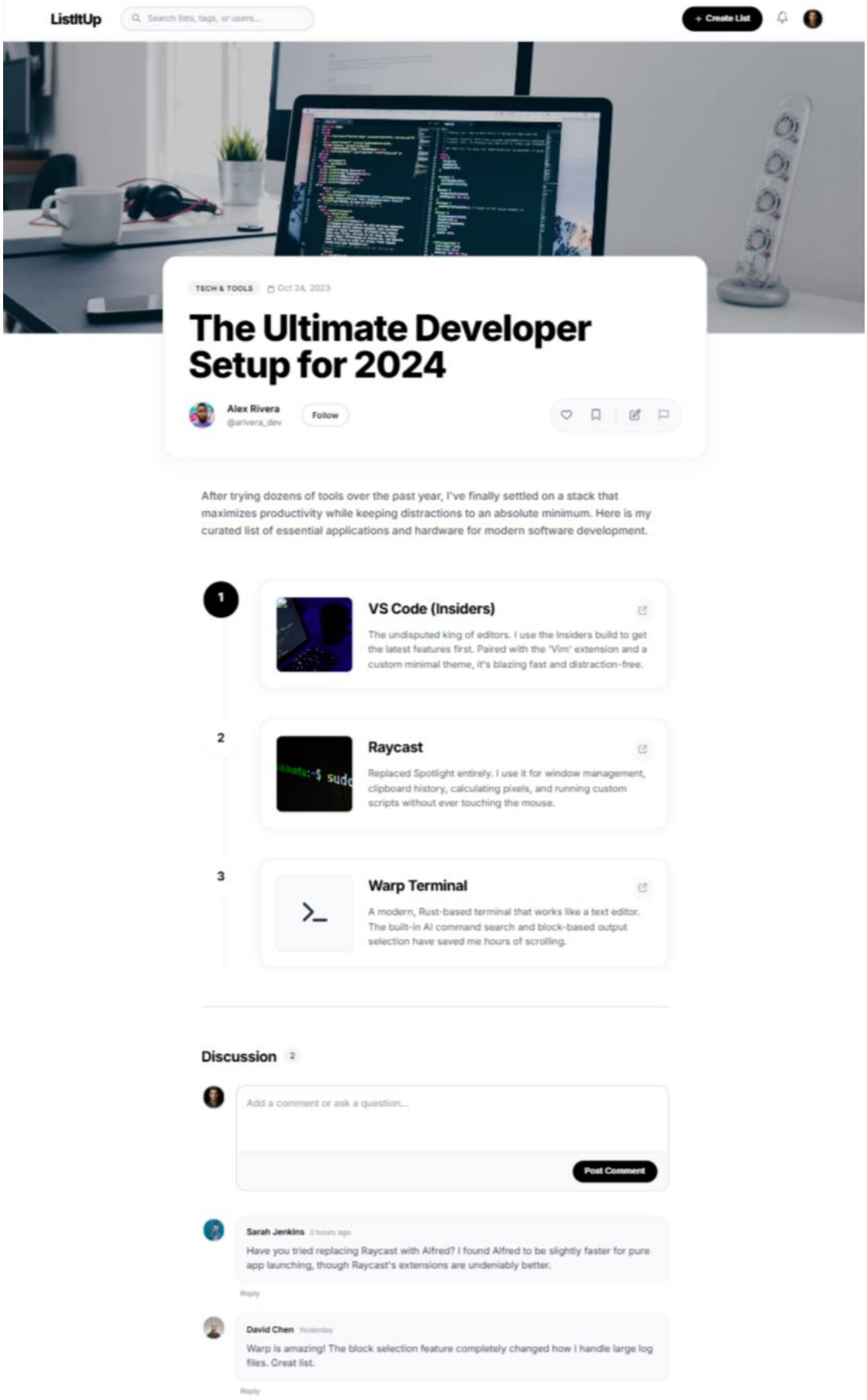
These are the different pages we designed for this project, but we do not discard the fact that we may need to make new ones as the project advances and evolves.

Screen 1 - Home Feed (/feed or /)



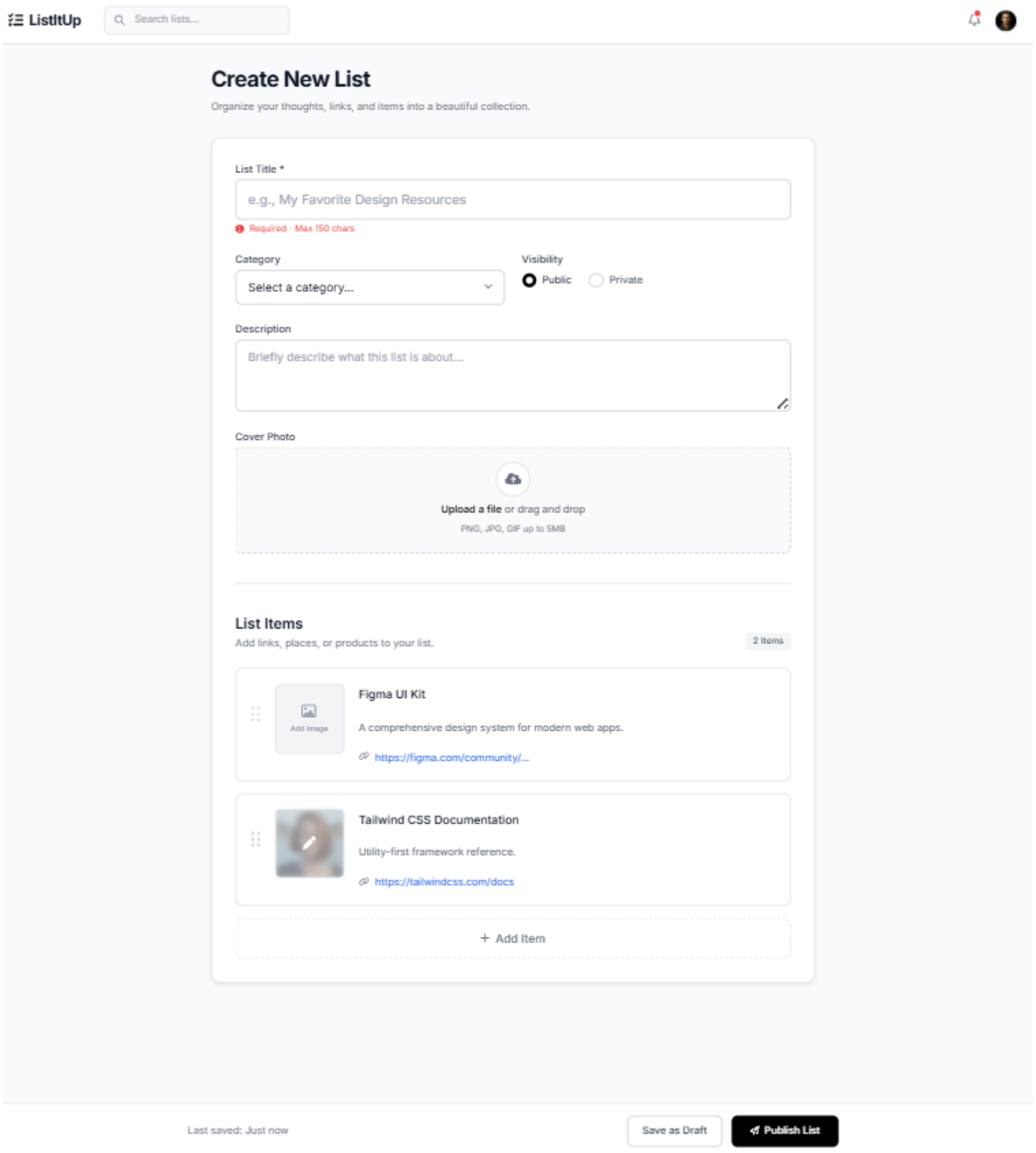
Component	Event	Behaviour
Category pill (e.g. 'Books')	click	Appends ?category=books to URL; page reloads with filtered feed. Active pill highlighted. Pills sourced from /categories endpoint.
Sort tab (Trending / Recent / Following)	click	Appends ?sort=trending|recent|following to URL; page reloads. 'Following' requires authentication; anonymous users are redirected to /login.
List card	click anywhere on card	Navigate to /lists/{listId}.
Heart icon on list card	click	JS fetch POST /lists/{listId}/like (toggle). Heart fills/unfills optimistically; reverts on error. Like count updates without page reload.
Notification bell (navbar)	click	Dropdown shows last 10 notifications. Unread badge clears. Each notification links to the related list.
Search bar (navbar)	submit (Enter / magnifier click)	Navigate to /search?q={keyword}.
'+ Create List' button (navbar)	click	Navigate to /lists/new. If not authenticated, redirect to /login with redirect-after-login param.

Screen 2 - List Detail (/lists/{listId})



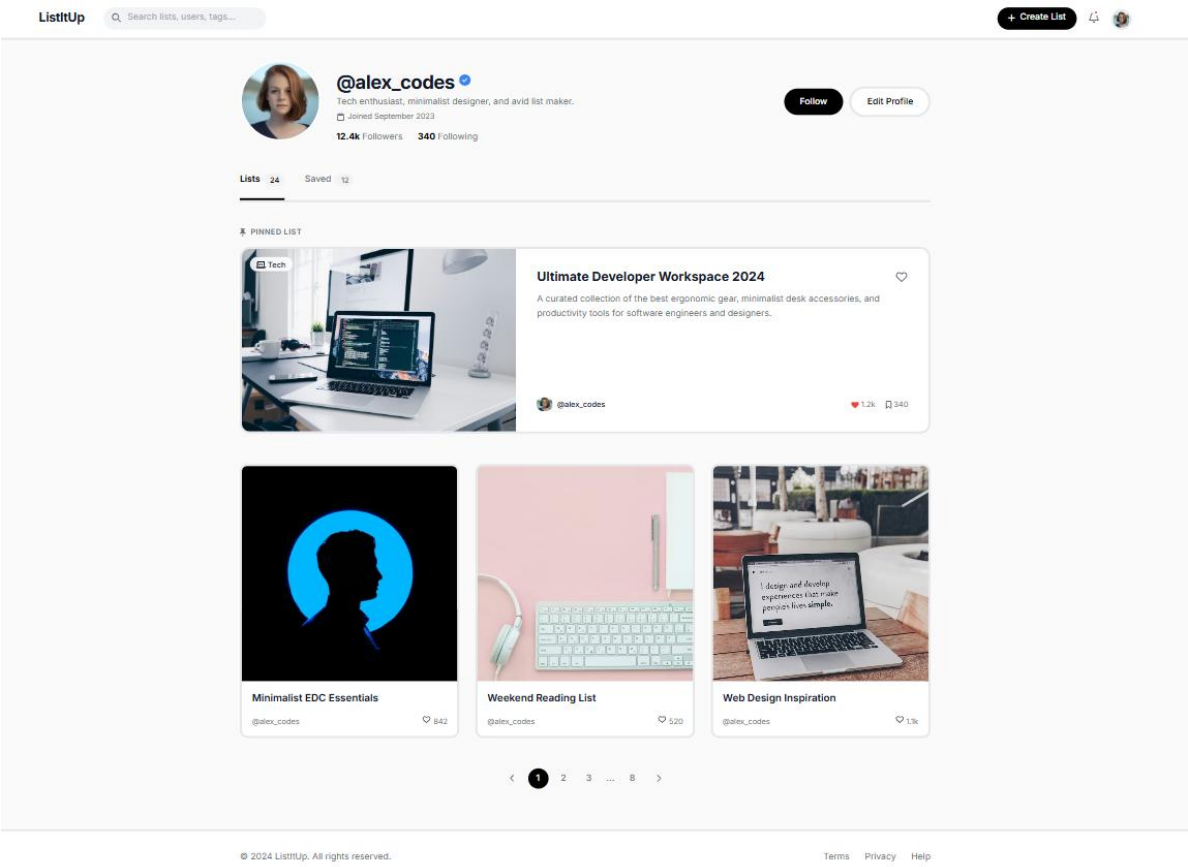
Component	Event	Behaviour
♥ Like button	click	Toggle via fetch POST /lists/{id}/like. Button state and count update without reload.
🔖 Save button	click	Toggle via fetch POST /lists/{id}/save. Redirects to /login if unauthenticated.
✎ Edit button (owner only)	click	Navigate to /lists/{id}/edit.
External link 🔗 on item	click	Opens URL in new tab. JS fires a link-click analytics event to /lists/{id}/items/{itemId}/click (fire-and-forget).
Comment textarea + Post button	click Post	AJAX POST /lists/{id}/comments with {text}. New comment card appended to DOM. Empty text rejected client-side.
🗑 Delete comment (author or admin only)	click	Confirm dialog → fetch DELETE /lists/{id}/comments/{commentId}. Card removed from DOM on success.
⚑ Report this list	click	Modal opens with reason textarea. On submit: POST /reports with {targetListId, reason}. Modal closes with success toast.

Screen 3 - Create / Edit List (/lists/new and /lists/{id}/edit)



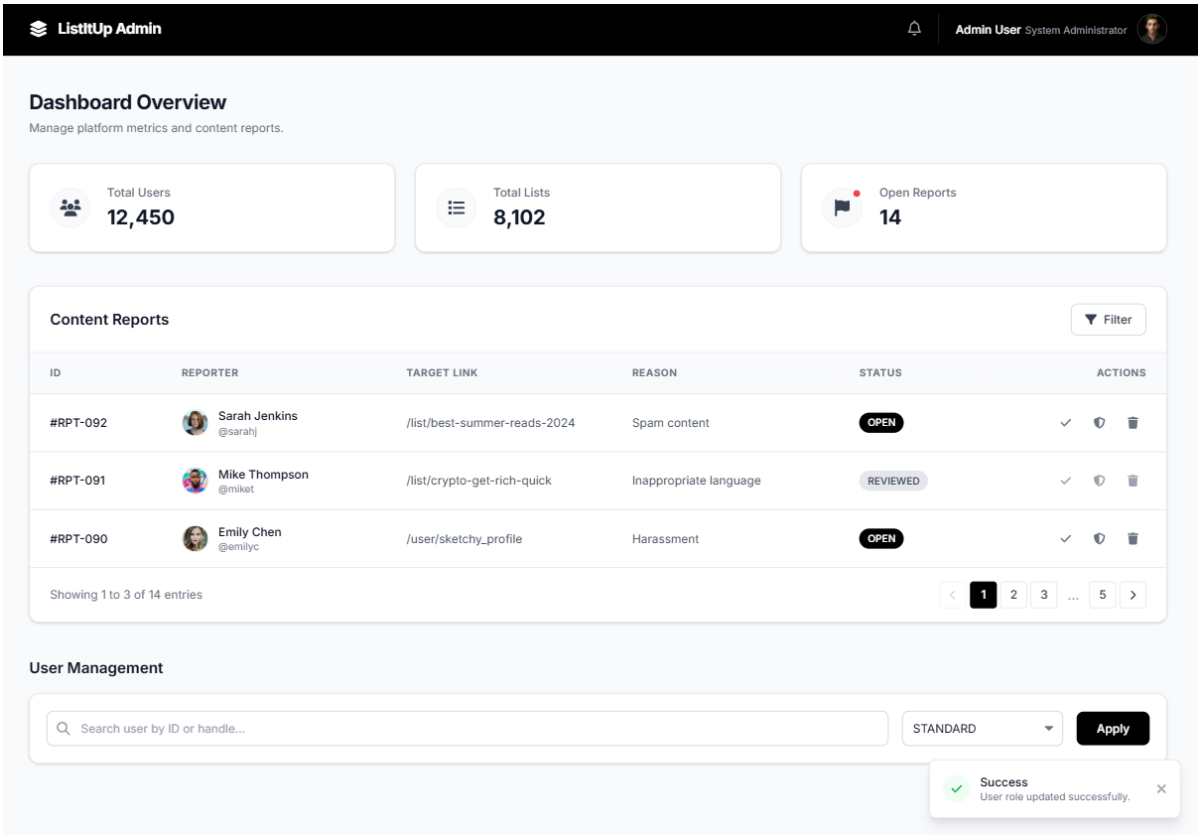
Component	Event	Behaviour
Title field	blur / input	Client-side validation: required, max 150 chars. Inline error displayed on violation.
Category dropdown	change	Populated via categories loaded on page render (Thymeleaf model).
Cover Photo upload	file selected	Preview shown below button. File uploaded via fetch POST /upload/image; returned URL stored in hidden form field.
Visibility radio	change	Toggles visual indicator. No server call until form submission.
Item drag handle (⠿)	drag-and-drop (HTML5 drag API)	Items reorder in DOM. On drop, positionIndex values recalculated and stored in hidden inputs.
Item URL field	blur	If URL non-empty, fetch GET /og?url={url}. Response auto-fills adjacent title/description fields if currently empty.
🗑 Delete item button	click	Remove item row from DOM. On edit, item marked for deletion via hidden delete_ids[] input.
+ Add item button	click	Appends a new empty item row. Max 50 items enforced client-side.
Publish button	click / form submit	Client validates: title required + ≥1 item with title. On pass: POST (create) or PUT (edit) to /lists[/{id}]. Server validates with Bean Validation. Success → redirect to /lists/{id}. Error → form re-rendered with messages (Thymeleaf).

Screen 4 - User Profile (/users/{username})



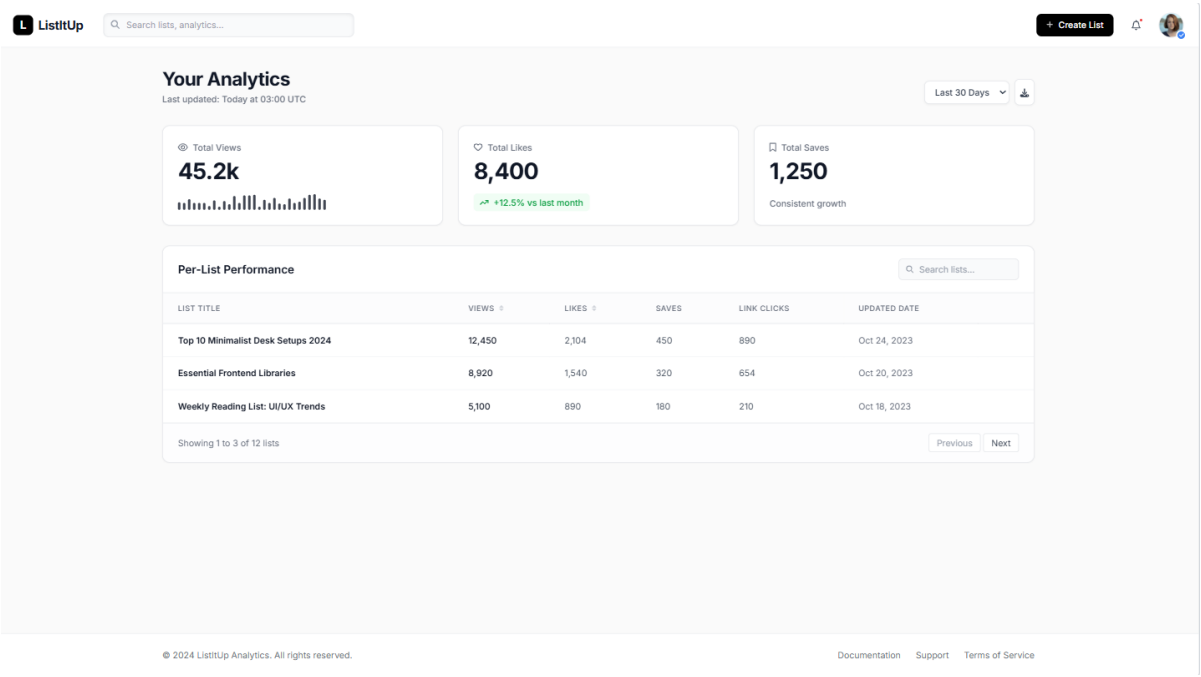
Component	Event	Behaviour
Follow / Unfollow button	click	fetch POST/DELETE /users/{userId}/follow. Button label toggles. Follower count updates without reload.
Edit Profile button	click	Navigate to /users/me/edit. Form with username, bio, avatar upload.
Saved tab	click	fetch GET /users/{username}?tab=saved. Shows lists the user has saved.
Follower / Following count link	click	Modal opens listing followers or following users with Follow/Unfollow buttons.
Pinned list card (Verified only)	click	Navigate to /lists/{listId}.

Screen 5 - Admin Panel (/admin (ADMIN role only))



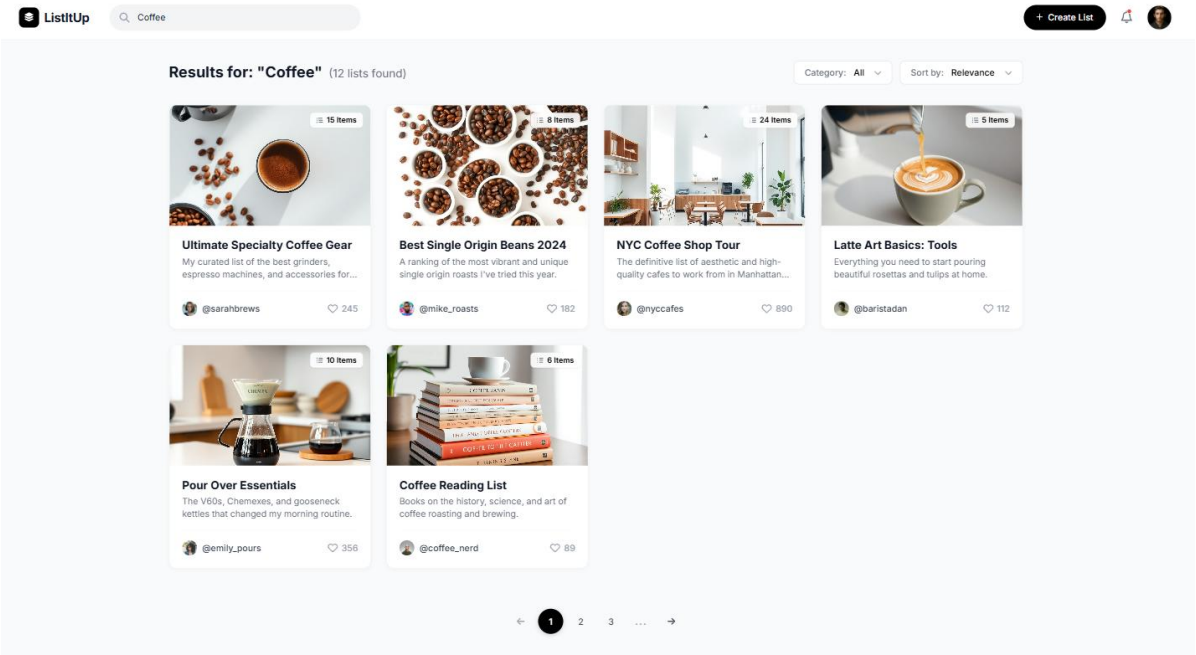
Component	Event	Behaviour
Mark Reviewed button (per report row)	click	fetch PATCH /admin/reports/{id} {status: 'REVIEWED'}. Row status badge updates inline.
Delete Content button (per report row)	click	Confirm dialog → fetch DELETE /admin/content/{type}/{id}. Removes list or comment; report auto-resolved. Row updates inline.
Role dropdown + Apply button	click Apply	fetch PATCH /admin/users/{id}/role {role: 'VERIFIED'|'STANDARD'|'ADMIN'}. Confirmation toast shown.
Spring Security access control	any request to /admin/**	If user.role != ADMIN: HTTP 403 returned; Thymeleaf renders 'Forbidden' page.

Screen 6 - Creator Analytics Dashboard (/analytics (VERIFIED+ only))



Component	Event	Behaviour
Per-list table column header (Views, Likes, etc.)	click	fetch GET /analytics?sort={column}&order={asc|desc}. Table re-renders sorted. Sort state stored in URL params.
List title link in table	click	Navigate to /lists/{id}.
Spring Security access control	any request to /analytics	If user.role == STANDARD: HTTP 403. Redirect to /feed.
Daily aggregation (scheduled task)	@Scheduled(cron = '0 0 3 * * *')	AnalyticsService aggregates view_count, like count, save count, link_click_count per list at 03:00 UTC daily. Results stored in analytics_snapshot table. Dashboard reads from snapshot for fast page load.

Screen 7 - Search Results (/search?q=&category=&sort=)



Component	Event	Behaviour
Category filter dropdown	change	Updates URL param ?category=; page reloads with filtered results.
Sort dropdown (Relevance / Recency)	change	Updates URL param ?sort=relevance|recency; page reloads.
Pagination controls	click page number	Updates URL param ?page=N; page reloads. Server returns Page<CuratedList> slice via Spring Data Pageable.